

REST APIs

Java Backend Interview — Short Notes

1. Fundamentals

- **REST (Representational State Transfer)** — Architectural style for distributed systems, not a protocol
- **REST vs SOAP** — REST is lightweight, uses JSON/XML, stateless; SOAP is protocol-based, XML-only, strict standards, WS-Security

HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Retrieve resource	Yes	Yes
POST	Create resource	No	No
PUT	Replace entire resource	Yes	No
PATCH	Partial update of resource	No*	No
DELETE	Remove resource	Yes	No
HEAD	Like GET but returns headers only — no body. Used to check existence or get metadata	Yes	Yes
OPTIONS	Returns allowed HTTP methods for the endpoint. Browser uses for CORS preflight	Yes	Yes

* PATCH can be idempotent depending on implementation

Statelessness & Idempotency

- **Stateless** — Server holds NO client session state. Each request must carry all context (auth, session info)
- **Idempotent** — Same request N times = same result (GET, PUT, DELETE, HEAD, OPTIONS)
- **Safe** — Read-only, no side effects (GET, HEAD, OPTIONS)

2. Status Codes

Code	Name	Meaning & When to Use
2xx — Success		
200	OK	Standard success. Use for GET, PUT, PATCH responses that return a body
201	Created	Resource created. Always include Location header pointing to the new resource URL
202	Accepted	Request accepted but processing not complete. Used for async operations — return a polling URL
204	No Content	Success but no body to return. Common for DELETE; sometimes PUT when not returning updated resource
3xx — Redirection		
301	Moved Permanently	Resource permanently moved to new URL. Clients/browsers should update cached URL for future requests
302	Found	Temporary redirect. Client should keep using original URL for future requests

304	Not Modified	Sent in response to conditional GET (If-None-Match). Body omitted — client uses its cached version
4xx — Client Errors		
400	Bad Request	Malformed request — bad JSON, missing required fields, invalid parameter types
401	Unauthorized	No valid credentials provided. Client should authenticate and retry. Name is misleading — it means unauthenticated
403	Forbidden	Authenticated but lacks permission. Don't reveal why — sometimes returning 404 is safer to hide resource existence
404		
405	Method Not Allowed	HTTP method not supported on this endpoint (e.g., DELETE on /users). Response must include Allow header
409	Conflict	State conflict — duplicate entry, optimistic locking failure, or version mismatch
410	Gone	Resource permanently deleted. Stronger signal than 404 — tells clients to stop requesting it
422	Unprocessable Entity	Syntactically valid but semantically wrong (e.g., end date before start date, invalid foreign key)
429	Too Many Requests	Rate limit exceeded. Always include Retry-After header so client knows when to retry
5xx — Server Errors		
500	Internal Server Error	Unexpected server failure. Never expose stack traces or internal details in response
502	Bad Gateway	Server acting as gateway received invalid response from upstream server
503		
504	Gateway Timeout	Upstream server didn't respond in time. Common in microservice chains when a downstream service is slow

3. URI Design Best Practices

- **Use nouns, not verbs** — /users not /getUsers (HTTP method expresses the action)
- **Plural resource names** — /orders not /order
- **Lowercase with hyphens** — /user-profiles not /UserProfiles
- **Hierarchical relationships** — /users/{id}/orders/{orderId}
- **Path params** — for resource identity: /users/{id}
- **Query params** — for filtering, sorting, pagination: /users?role=admin&page=1&size=20
- Keep URIs consistently lowercase — they are case-sensitive by spec
- Avoid deep nesting beyond 2 levels — prefer /orders?userId=1 over /users/1/orders/5/items

4. Request & Response

Request Headers

Header	Purpose & Example
--------	-------------------

Content-Type	Format of the request body. e.g., application/json, multipart/form-data, application/x-www-form-urlencoded
Accept	Format the client expects in response. e.g., Accept: application/json. Server returns 406 if it cannot match
Authorization	Credentials for the request. e.g., Bearer or Basic base64(user:pass)
Cache-Control	Caching directives sent by client. e.g., no-cache (always revalidate with server), max-age=3600, no-store (never cache)
If-None-Match	Conditional GET — sends the ETag value received earlier. Server returns 304 Not Modified if resource unchanged, saving bandwidth
If-Modified-Since	Conditional GET based on timestamp. Server returns 304 if resource not modified since the given date
Accept-Language	Client preferred language for response. e.g., en-US, fr. Server uses this for localized content
Accept-Encoding	Compression formats supported by client. e.g., gzip, br (Brotli). Server compresses response accordingly
X-Request-ID	Unique ID per request for distributed tracing. Passed and logged across all microservices to trace one request end-to-end
X-Correlation-ID	Used to correlate multiple related requests in a workflow. Similar to X-Request-ID but for a chain of calls
X-Forwarded-For	Original client IP when request passes through proxy or load balancer. e.g., X-Forwarded-For: 203.0.113.1
Idempotency-Key	Client-generated unique key for safe POST retries. Server deduplicates using this key — critical for payment requests

Response Headers

Header	Purpose & Example
Content-Type	Format of the response body. e.g., application/json; charset=UTF-8
ETag	Resource version hash. Client stores this and sends back in If-None-Match on next request to enable caching
Location	URL of newly created resource (with 201) or redirect target. e.g., Location: /api/users/123
Retry-After	Seconds or date to wait before retrying. Sent with 429 (rate limit) and 503 (service unavailable)
WWW-Authenticate	Tells client what auth scheme to use. Sent with 401. e.g., WWW-Authenticate: Bearer realm='api'
Cache-Control	Caching directives for the response. e.g., public, max-age=3600 (CDN can cache) or private, no-cache
Sunset	Date when this API version will be deprecated. e.g., Sunset: Sat, 31 Dec 2025 00:00:00 GMT
X-Rate-Limit-Remaining	How many requests the client has left in the current rate limit window
Access-Control-Allow-Origin	CORS header specifying which origins can access the resource. e.g., * or https://myapp.com

Content Negotiation

- Client sets **Accept** header; server responds in matching format or returns **406 Not Acceptable**

Standard Error Response Structure

```
{ "timestamp": "2024-01-01T10:00:00Z", "status": 400, "error": "Bad Request",  
  "message": "Field email is required", "path": "/api/users" }
```

Pagination Pattern

- Request: `GET /users?page=0&size=20&sort=name,asc`
- Response must include: `data` array, `totalElements`, `totalPages`, `currentPage`, `hasNext`

5. Versioning

Strategy	Example	Pros	Cons
URI Versioning	<code>/v1/users</code>	Simple, visible, cacheable	Pollutes URL
Header Versioning	Accept: <code>application/vnd.app.v1+json</code>	Clean URIs, no URL change	Less discoverable, harder to test in browser
Param Versioning	<code>/users?version=1</code>	Easy to test in browser	Messy query params

- **URI versioning** is most widely used in practice
- Deprecate old versions with **Sunset** header and communicate timelines
- Distinguish **breaking** (new major version) vs **non-breaking** changes (add fields, new endpoints)

6. Authentication Mechanisms

Basic Auth

- Base64 encode **username:password** and send in Authorization header on every request
 - Simple but credentials sent each time — only safe over HTTPS. No token expiry
- Example: Authorization: Basic dXNlcjpwYXNz — used in internal tools, admin panels, dev environments. Never over plain HTTP.*

API Keys

- Static secret key passed in header (**X-API-Key**) or query param. No expiry by default
 - Good for service-to-service. No user context — rotation must be managed manually
- Example: X-API-Key: abc123xyz — used by Stripe, Google Maps, Twilio. Revoke and rotate if compromised.*

Session-based Auth

- User logs in → server creates session (stored in DB/Redis) → returns **Set-Cookie: sessionId=xyz**
 - Every request carries cookie. Stateful — scales poorly without sticky sessions or shared session store
- Example: Traditional web apps (banking portals). Works in single-server setups; needs Redis in clustered environments.*

JWT (JSON Web Token)

- Format: **header.payload.signature** — Base64 encoded, signed with secret or private key
- Stateless — server verifies signature without any DB lookup on each request
- **Access token** — short-lived (5–15 min). **Refresh token** — long-lived (days), used only to get new access token
- Storage: **HttpOnly cookie** preferred (XSS-safe) over `localStorage` (vulnerable to XSS)
- Weakness: can't be invalidated before expiry — mitigate with short expiry + token blacklist

Example: Login returns JWT. Client sends Authorization: Bearer . Server validates signature — no DB call needed. Used in most modern APIs and mobile apps.

OAuth2

- Delegated **authorization** framework — allows third-party limited access without sharing credentials
- **Roles:** Resource Owner (user), Client (app), Authorization Server, Resource Server
- **Grant Types:**
 - **Authorization Code** — Most secure; for server-side web apps
 - **PKCE** — Authorization Code for SPAs and mobile apps; prevents code interception attacks
 - **Client Credentials** — Machine-to-machine; no user involved
 - **Implicit** — Deprecated; was used for SPAs

Example: User clicks 'Login with Google' → redirected to Google → approves → Google sends auth code → your server exchanges it for an access token → calls Google APIs on user's behalf.

OpenID Connect (OIDC)

- Identity layer on top of OAuth2 — adds **authentication** (OAuth2 handles only authorization)
- **ID Token** — JWT with user identity claims (name, email, sub). **Access Token** — for resource access

Example: After 'Sign in with Google', ID token contains name and email — used for SSO across apps (Google, Apple, Microsoft login).

mTLS (Mutual TLS)

- Both client AND server authenticate via certificates — bidirectional trust, no passwords
- Common in zero-trust microservice meshes (Istio, AWS service mesh)

Example: Payment service presents its cert to fraud detection service; fraud service presents its cert back. Both verify before any data flows.

7. Best Practices

- **Idempotency keys** — For POST, clients send unique key in header; server deduplicates to prevent double processing
- **Rate limiting** — Return 429 with Retry-After header. Use token bucket or sliding window algorithm
- **CORS** — Configure allowed origins; browser sends OPTIONS preflight for cross-origin requests
- **HATEOAS** — Include links to related actions in response; theoretically pure REST, rarely used in practice
- **OpenAPI/Swagger** — Document APIs; enables auto-generated clients and interactive testing (Swagger UI)
- **Async APIs** — For long operations, return 202 Accepted + polling endpoint or use webhooks
- **File uploads** — Use multipart/form-data for small files; streaming or S3 presigned URLs for large files
- **Sensitive data** — Never in URLs (URLs get logged); use headers or body; mask in logs; HTTPS always

8. Frequently Asked & Tricky Interview Questions

Fundamentals

Q1. What is REST and what are its 6 constraints?

→ Stateless, Client-Server, Cacheable, Uniform Interface, Layered System, Code on Demand (optional)

Q2. What's the difference between REST and SOAP?

→ REST = architectural style, lightweight, JSON/XML, stateless. SOAP = protocol, XML-only, strict standards, WS-Security

Q3. What's the difference between REST and GraphQL?

→ REST = multiple endpoints, fixed structure. GraphQL = single endpoint, client specifies exact fields needed

Q4. Is REST a protocol or architectural style?

→ Architectural style — not a protocol

Q5. What does stateless mean in REST?

→ Server holds NO client session state. Each request must carry all context needed (auth, session info)

Q6. Can GET have a body?

→ Technically yes, but strongly discouraged — proxies/servers may ignore it

Q7. What is HATEOAS and is it practical?

→ Hypermedia As The Engine Of Application State — responses include links. Theoretically pure REST; rarely practical

Q8. What is content negotiation?

→ Client sends Accept header specifying desired format; server responds accordingly or returns 406

Q9. Difference between Content-Type and Accept?

→ Content-Type = format I'm sending. Accept = format I want back

Q10. What is idempotency? Which methods are idempotent?

→ Same request N times = same result. GET, PUT, DELETE, HEAD, OPTIONS are idempotent. POST and PATCH are not by default

HTTP Methods

Q11. Difference between PUT and PATCH?

→ PUT replaces the entire resource. PATCH updates specific fields only

Q12. When would you use PATCH over PUT?

→ When updating partial data — e.g., changing only email without sending full user object

Q13. What does HEAD return?

→ Same headers as GET but no response body. Use to check resource existence or get Content-Length

Q14. What is OPTIONS used for?

→ Returns allowed HTTP methods. Browser uses it for CORS preflight requests

Q15. Can POST be idempotent?

→ Not by default, but can be made idempotent using Idempotency-Key headers

Q16. Why is DELETE idempotent?

→ Deleting an already-deleted resource still results in the same state — resource doesn't exist

Q17. Is GET always safe? What does safe mean?

→ Safe = no side effects, read-only. GET is safe by spec, but a poorly implemented GET could trigger changes

Status Codes

Q18. Difference between 401 and 403?

→ 401 = not authenticated (who are you?). 403 = authenticated but not authorized (I know you, but you can't do this)

Q19. When to use 400 vs 422?

→ 400 = malformed request (bad JSON). 422 = well-formed but semantically invalid (missing required field, invalid date range)

Q20. Difference between 301 and 302?

→ 301 = permanent redirect (update your links). 302 = temporary redirect

Q21. When to return 204?

→ Successful operation with no body — common with DELETE, sometimes PUT

Q22. What does 409 Conflict mean?

→ Request conflicts with current state — e.g., creating a duplicate resource or optimistic locking failure

Q23. Difference between 500 and 503?

→ 500 = unexpected server error. 503 = server intentionally unavailable (maintenance/overload)

Q24. 201 vs 200 for POST?

→ 201 = resource created (preferred for POST). Include Location header pointing to new resource

URI Design

Q25. Why use nouns not verbs in URIs?

→ HTTP methods already express the action. /users + DELETE is cleaner than /deleteUser

Q26. When path params vs query params?

→ Path = resource identity (/users/123). Query = filtering/sorting/pagination (/users?active=true)

Q27. How to represent relationships in URIs?

→ /users/{id}/orders — but avoid nesting beyond 2 levels

Q28. Should URIs be case-sensitive?

→ By convention, keep them lowercase. RFC says path is case-sensitive, but practice prefers consistent lowercase

Versioning

Q29. Which versioning approach is best?

→ URI versioning is most widely adopted due to simplicity, visibility, and cacheability

Q30. Pros/cons of URI vs header versioning?

→ URI = simple, cacheable, visible but pollutes URL. Header = clean URLs but less discoverable, harder to test

Q31. How do you deprecate an API version?

→ Use Sunset header, communicate timelines, maintain old version during transition period

Security

Q32. Difference between authentication and authorization?

→ AuthN = who are you? AuthZ = what are you allowed to do?

Q33. How does JWT work internally?

→ header (algo) + payload (claims) + signature (HMAC/RSA). Server verifies signature without DB lookup

Q34. Where to store JWT — localStorage vs cookie?

→ HttpOnly cookie preferred — immune to XSS. localStorage is vulnerable to XSS attacks

Q35. Access token vs refresh token?

→ Access = short-lived (15min). Refresh = long-lived (7d), stored securely, used only to get new access token

Q36. Difference between OAuth2 and OIDC?

→ OAuth2 = authorization only. OIDC = adds authentication layer on top, provides ID token with user identity

Q37. OAuth2 grant types and when to use?

→ Auth Code = web apps. PKCE = SPAs/mobile. Client Credentials = M2M. Implicit = deprecated

Q38. What is mTLS?

→ Both parties present certificates. Common in microservices for zero-trust security

Q39. How to prevent replay attacks?

→ Short token expiry, use nonces, HTTPS only, token binding

Q40. What is CORS?

→ Cross-Origin Resource Sharing — browser security mechanism. Server must explicitly allow cross-origin requests

Q41. What is a preflight request?

→ OPTIONS request browser sends automatically before cross-origin POST/PUT to check server permissions

Q42. How to secure sensitive data in responses?

→ Never in URLs, mask in logs, encrypt at rest, use HTTPS, filter response fields by role

Best Practices & Design

Q43. How to design pagination?

→ Offset-based (?page=0&size=20) for simple use; cursor-based for large or real-time data sets

Q44. What if a REST API is not idempotent where it should be?

→ Duplicate side effects — e.g., double charging, double creating. Use idempotency keys to fix

Q45. Standard structure for error response?

→ Include: timestamp, status, error, message, path. Optionally: errorCode, details array

Q46. Breaking vs non-breaking API changes?

→ Non-breaking = add optional fields, new endpoints. Breaking = remove/rename fields, change types — needs new version

Q47. How to rate-limit a REST API?

→ Return 429 Too Many Requests with Retry-After header. Use token bucket or sliding window algorithm

Q48. Sync vs async REST APIs?

→ Sync = immediate response. Async = return 202 + polling endpoint or webhook for long operations

Q49. How to handle file uploads?

→ multipart/form-data for small files; streaming for large files; consider S3 presigned URLs

Q50. How to document a REST API?

→ OpenAPI/Swagger spec — enables auto-generated clients, interactive docs (Swagger UI), validation